NANODISCOS MAGNÉTICOS · SECCIÓN 0

MCMC en GPU: por qué el número de réplicas cambia el costo, y qué se hizo al respecto

Informe completo de la sección: lo que se buscó, lo que se midió, cada hipótesis que se descartó y cada error que se cometió. Es el registro largo; las seis páginas interactivas de las que sale quedan en `paginas/`.

Generado el 2026-10-23 · repositorio `canamejoy/Spin-State-GenerationAndInference` · todas las cifras provienen de los JSON en `resultados/`

Contenido

1. El objeto de estudio — p. 2
2. El hallazgo: la regla del ancho de vector — p. 3
3. El dentado — p. 6
4. Todo lo que se descartó — p. 7
5. Las mitigaciones sin kernel propio — p. 10
6. El kernel propio — p. 11
7. Metodología, y los errores que costó aprenderla — p. 16
8. Los números finales — p. 19
9. Cuántas réplicas hacen falta: τ, y la convención — p. 22
10. Registro de errores y correcciones — p. 23
11. Lo que queda abierto — p. 30
12. Inventario de la sección — p. 31

1 · El objeto de estudio

1.1 La física

Un nanodisco magnético: una red cúbica de espines clásicos recortada en forma de disco. La configuración de referencia es `Rd = 18.30` de radio y `L = 5` capas, lo que da `N = 39` de lado y **5245 sitios activos** dentro del disco de 7605 de la caja.

El Hamiltoniano lleva cinco términos, y la fidelidad a la referencia del proyecto importa porque uno de ellos es fácil de equivocar:

término	forma	nota
Intercambio	$-J \Sigma\, s \cdot s'$	primeros vecinos; las capas 2 y 4 son cero, que es lo que hace exacto el tablero de ajedrez
DMI **interfacial**	solo enlaces en el plano x e y, con signo menos	**no es intercambiable con la forma de todos los ejes: sobre la misma configuración la energía total difiere un 50 %**
Anisotropía cúbica	$K_1/S^4 \cdot \Sigma(s_i s_j)^2 + K_2/S^6 \cdot (s_x s_y s_z)^2$	serie K_1/K_2 de libro sobre cosenos directores; confirmado contra `sim_core` a precisión de máquina
Anisotropía de superficie	`K_anS` donde el sitio tiene menos de seis vecinos en el disco	es un campo por sitio, no un escalar
Campo externo	$-H \cdot s$	ángulo `g`, entra como `cos g` y `sin g`

1.2 El algoritmo

Metropolis de un solo sitio con propuesta uniforme sobre la esfera, pero actualizando **por tablero de ajedrez**: los sitios se colorean por `(x+y+z) mod 2` y se actualiza un color completo a la vez. Con solo primeros vecinos eso es un **reordenamiento exacto** del barrido secuencial, no una aproximación — y es la propiedad que más tarde permite leer el halo sin sincronización.

Un barrido son dos medios pasos (par, impar). El cronograma de producción es **200 temperaturas** de 20.0 a 0.1 K, con **20 000 barridos** por temperatura repartidos dos tercios en termalización y uno en medición: en total **4 000 000 de barridos por réplica**.

> **Por qué se mide en µs por réplica-barrido y no en reloj pelado.** El reloj mezcla dos cosas: cuántas réplicas hay y cuánto cuesta cada una. Un lote de 1024 réplicas tarda más que uno de 10 y es mucho más eficiente. Dividir por réplicas y por barridos aísla la eficiencia, que es lo que cambia. En este informe se dan las dos cosas, porque la tasa es la que compara y el reloj es el que se paga.

1.3 Qué se paraleliza

Hay tres ejes posibles y no valen lo mismo:

eje	ancho	veredicto
Dentro de una réplica (el tablero)	~2600 sitios por color	obligatorio; es corrección, no velocidad
Réplicas	lo elegimos nosotros	**el único eje ancho, y el objeto de este estudio**
Estados (conjuntos de parámetros)	pocos, los fija la física	entran como una columna que se difunde, no como una ramificación: cuatro estados en un lote cuestan lo mismo que 100 réplicas de uno (**+0.3 %**)

> **Y 100 réplicas nunca fue un límite de hardware.** Era el número que alguien eligió. El límite real es el rendimiento, y ahí empieza el problema de esta sección.

2 · El hallazgo: la regla del ancho de vector

2.1 La curva no es convexa

La primera hipótesis era la obvia: existe un número óptimo de réplicas por GPU, y la curva de costo tiene forma de U. Al medirla con ventanas **contiguas** de lote —y eso es lo único que hace visible la estructura fina— apareció algo distinto: la curva es **dentada**. Diecinueve

puntos, el mínimo en `B = 512`, y su vecino inmediato un **57 % más caro**. No se puede interpolar.

En un barrido parejo de cincuenta lotes el mejor fue `B = 850` con **7.33 s por réplica** en una corrida completa, y el peor **15.22 s**: más del doble, con el mismo trabajo y la misma física.

2.2 La regla

La estructura fina no es ruido. Depende de la divisibilidad por dos del número de elementos del arreglo. Con $v_2(n)$ = cuántas veces n se divide por dos:

$$\text{elementos} = B \cdot L \cdot N^2 \cdot 3 \quad \Rightarrow \quad \text{ancho} = \min(4,\ 2^{v_2(B) + v_2(L)})$$

Como N es siempre impar, $v_2(\text{elementos}) = v_2(B) + v_2(L)$. Hay castigo cuando $v_2(B) + v_2(L) \geq 2$. **89 de 89 rejillas predichas sin un fallo.**

Y «múltiplo de cuatro» significa cosas opuestas para `B` y para `L`. Que `B` lo sea dispara el costo alto en ese lote. Que `L` lo sea vuelve la red indiferente al lote — pero pagando el costo alto en todas las celdas. La inmunidad compra previsibilidad, no velocidad.

Medido: en la T4, `L` divisible por cuatro está estrictamente dominado — peor en el mejor caso, peor en el esperado y peor en el peor caso. En la H100 es un canje estrecho: se entrega 1.9 % de valor esperado para recortar la varianza.

2.3 El mecanismo

XLA le da a cada hilo **cuatro elementos consecutivos** en vez de uno. Un warp pide entonces 512 bytes para entregar 128 útiles: rompe la coalescencia de memoria. El volcado de PTX lo confirma — `ld.global.v4 = 0`, o sea que no son cargas vectoriales de 128 bits sino **cuatro cargas escalares** por hilo.

Se reprodujo **en CUDA puro, sin compilador**: el reparto consecutivo cuesta **+14.9 %** y el kernel de JAX paga **+14.25 %**.

> **Esto corrige una narrativa anterior. «Es el compilador, no el hardware» era una falsa dicotomía. XLA toma la decisión —qué ancho, y consecutivo en vez de escalonado— y el sistema de memoria pone el precio. Por eso el castigo correlaciona con la tarjeta.**

Y hay un umbral de huella. La regla no se enciende por debajo de unos **5 MB** de conjunto de trabajo. Un experimento anterior fijó los sitios en 2125 creyendo controlar la variable correcta, y con eso fijó también la huella en 2.1 MB — midiendo en un régimen **donde el efecto no existe**. Ver el registro de errores.

2.4 No se puede apagar

Si existiera un flag para desactivar la vectorización no habría regla que encontrar. Se volcó la lista completa de flags del build instalado: de los **16 flags** `xla_gpu`, **ninguno** controla la vectorización. `--xla_cpu_prefer_vector_width` existe, pero solo para CPU.

Forzando un elemento por hilo el castigo desaparece (−0.07 %) y `B = 64` pasa a ser el más rápido — pero eso exige reescribir el kernel, que es precisamente lo que acabó haciéndose en la sección 6.

2.5 Alcance verificado

afirmación	dónde está medida
El costo alto existe en los múltiplos de cuatro	T4, L4, A100, H100, H200 — **5 GPU, 4 arquitecturas** (+42 / +38 / +21 / +12 / +14 %)
Las capas deciden (`v₂(L)`)	H100 y T4 — 2 arquitecturas
La geometría que emite el compilador	**89 de 89 rejillas** predichas sin fallo
La causa es la coalescencia, no el ancho en sí	CUDA puro: +14.9 % contra +14.25 % de JAX
El ancho de banda **no** es la variable	control en H200: mismos 132 multiprocesadores, **1.43×** el ancho de banda, y el castigo **no bajó** (+14.0 % contra +12.1 %)

3 · El dentado

Encima del ancho de vector hay una segunda estructura: dos lotes **del mismo ancho predicho** pueden diferir un **20 %**, y en H100 se midió una ondulación de **27 a 35 %** entre lotes adyacentes. Esto es lo que impide tratar la curva como suave.

3.1 El mecanismo, hallado

La fusión que lleva el dentado es de tipo `kCustom` con `"kind":"__triton"`: XLA delega ese kernel a Triton, que **re-teja el kernel para cada forma** con su propio modelo de costo.

La rejilla es `grid = Π ⌈dim/tile⌉`, **predicha 8 de 8**. Y el tile no es fijo: `tile_B` toma valores 4, 8, 16, 32 y hasta `B` según el lote.

Del modelo de costo se derivó además una restricción: el costo cuenta sectores de 32 bytes, un tile escribe `3·tx` flotantes contiguos, y de ahí `tx ≡ 0 (mod 8)` — verificado **18 de 18**, con `tx ∈ {8,16}` únicamente. Con el presupuesto de 1536 elementos (`tb·ty·tx = 512`) quedan solo **12 candidatos**.

Pero no tiene forma cerrada, y eso costó tres reglas falsas. Se intentó predecir qué tile elige Triton:

- **`B mod 4` — retirada: asumía `tile_B` = 4 siempre, y varía.**
- **`B mod 8 ∈ {5,7}` — pre-registrada en git antes de medir, puntuó 9 de 12 y murió.**
- **`B mod 16` — refutada con datos ya en disco (contraejemplos en B = 45, 47, 75, 77, 79, 91, 93, 95), sin gastar una corrida.**

La conclusión operativa es que un candidato se elige compilando y leyendo la firma de tiles, no con una regla. Validado a escala de producción: +15.0 % entre dos lotes que la firma distingue.

3.2 Dónde se va el tiempo, por HLO

operación	fracción del anneal
Los seis `roll` (alinear vecinos)	**31.5 %**
Threefry (el generador aleatorio)	**17.1 %**
La física propiamente dicha	20.3 %
Resto (reducciones, acumuladores, scan)	31.1 %

Un solo kernel — `fusion_1458` — explica el **67.8 %** de la diferencia entre `B = 65` y `B = 75`, y es el único cuya rejilla no escala proporcionalmente. Ese reparto es lo que después justificó atacar los `roll` y el RNG primero.

4 · Todo lo que se descartó

Esta es la parte larga del trabajo. Cada fila es una hipótesis que parecía razonable, con la medición que la cerró.

hipótesis	qué la cerró	veredicto
Hay un **número óptimo** y la curva es convexa	19 puntos contiguos: mínimo en 512, vecino +57 %	**falsa — la curva es dentada**
Lo que importa es **B × sitios** (el producto)	se pidió que seis posiciones coincidieran en tres geometrías; en `Rd = 24` la predicción salió **invertida las seis veces**, en `Rd = 12` acertó tres de seis	**falsa — la curva no se transfiere entre redes**
Es el **hardware**	la forma se midió dentro de un dispositivo con identificador auditable; la variación entre dispositivos está acotada en **1.5 %**; para explicar un salto del 57 % harían falta dispositivos 38× más dispares	**falsa**
Es el **ancho de banda** de memoria	H200 contra H100: mismos 132 multiprocesadores, **1.43×** el ancho de banda, castigo igual (+14.0 % contra +12.1 %)	**falsa**
Es el **número de multiprocesadores**	correlación **−0.998** sobre cinco dispositivos y gana 7 de 9 comparaciones por pares	**sobrevive como correlación, no como mecanismo: cinco puntos con cuatro variables colineales no bastan**
Es el **paso entre réplicas** en bytes	la regla `v₂(paso) ≥ 4 ⇒ inmune` predice las ocho geometrías, pero `v₂(paso) = 2 + v₂(L)`, así que decía `L` divisible por cuatro con otras palabras	**cierta y por la razón equivocada**
Es la **transferencia** CPU→GPU	por construcción: `pack()` devuelve `jnp.asarray`, `init_spins` devuelve un arreglo de dispositivo y una semilla de	**falsa, por dos caminos independientes**

hipótesis	qué la cerró	veredicto
	JAX ya lo es — no hay copia dentro de la región medida	
Es la **cuantización de olas**	medido: el costo de un lanzamiento es **~122 µs plano** entre 1 y 597 bloques; no hay escalones	**falsa**
Son conflictos de **canal de memoria** / partition camping	el mecanismo es de libro pero NVIDIA lo declara muerto después de Fermi; el trabajo más cercano es el de L2 no uniforme en la L40	**sin evidencia propia**
Es el **emisor de reducciones** o el costo de lanzamiento	descartados en el barrido de hipótesis de `lotes.html`	**falsas**
El **desperdicio de carriles** explica el costo del kernel propio	aplanar a 2048 redujo el desperdicio a la mitad; se predijo **−50 %** y se midió **−2.9 %**	**falsa — el costo es el direccionamiento**
XLA emite **conjuntos de kernels distintos** para lotes distintos	el volcado de PTX propio: firmas idénticas (`ld_scalar = 69` , `st = 16`)	**falsa — son los mismos kernels renombrados**
El eje de **estados** aporta paralelismo	leído en el código: `pack` + `shard` ponen **un estado por GPU**	**falsa**
Dos `pmap` sobre pares de GPU **disjuntos** se solapan	medido: **+83.8 %** — serializan	**falsa**

5 · Las mitigaciones sin kernel propio

5.1 Elegir el lote

Con `L` impar, mantener `v₂(B) + v₂(L) < 2` es exactamente **evitar los múltiplos de cuatro**, y cuesta cero: es cambiar un número entero. En la geometría de referencia y en H100 el recomendado es `B = 850` (7.33 s/réplica); `B = 705` cuesta 5 % más y cualquier lote en [385, 465] entre 9 % y 13 % más.

> **Y apuntar a una meseta, no a un punto. En la H100 los lotes 1025, 2049 y 4097 difieren solo 2.2 % entre sí. Un punto exacto es frágil frente al dentado; una meseta no.**

5.2 El protocolo de partición

Si el número que se necesita es múltiplo de cuatro, se parte en `B/2+1` y `B/2−1`: dos olas, las dos impares, la suma exacta. Medido sobre 4 H100 con el kernel de producción:

por dispositivo	partición	ganancia
400 (1 GPU)	201 + 199	**−28.3 %**
200 (2 GPU)	101 + 99	**−21.7 %**
100 (4 GPU)	53 + 47	**−26.8 %**

Dos olas con las mitades impares gana en los tres brazos, y **más olas empeora**: el lote por dispositivo cae a la zona empinada de `a/B`, con `a ≈ 122 µs` por lanzamiento. Confirmado en escalera completa de 4 GPU con observables incluidos: **−27.0 %**, contra el −26.8 % medido solo con el paso.

Y gana por lanzamientos, no por ancho. `51+49` **empata** con `50+50`, así que el ancho 2 es gratis. Una sola ola de 99 o 101 gana 8–10 % a cualquiera de los dos. Lo que el protocolo compra es evitar el ancho 4, no el ancho 2.

5.3 Lo que se intentó y no funcionó

intento	resultado	por qué
La capa tonta: añadir una capa enmascarada para cambiar `v₂(L)`	**6 de 6 celdas pierden, media −20.6 %**	la física es exacta (energía a 0.00e+00) pero una capa extra cuesta `1/L`, o sea 25 % en `L = 4`: cuesta lo mismo que el ancho ahorra
Elegir `L` divisible por cuatro para ser inmune	**estrictamente dominado en T4**	la inmunidad consiste en pagar ancho 4 en **todas** las celdas; sirve solo si el requisito es varianza cero
Rellenar la red hasta una potencia de dos	**peor dos veces**	más sitios que calcular, y el relleno no cambia el ancho
Optimización bayesiana del lote	**se perdió el óptimo: `B = 435` (8.01 s) contra `B = 850` (7.33 s), 9.3 % peor**	el modelo ya incorporaba la regla del ancho, así que el residuo era el dentado — que no tiene estructura que un modelo suave capture. Dejó además un coeficiente mal estimado: reportó +48.8 % de recargo para el ancho 4 a partir de **3 puntos**, los tres en los extremos

6 · El kernel propio

Si la decisión es del compilador y no es configurable, la salida de fondo es **no entregarle el bucle**. Eso significa escribir el kernel a mano.

6.1 Tres intentos fallidos en Pallas

intento	resultado	por qué
Pallas sobre `mosaic_gpu`	**+15 a +31 %**	ocho rechazos distintos del backend; el único tile admisible es **degenerado** (1×1), así que cada sitio se trae siete veces
Pallas sobre Triton, rejilla 2-D	**+26.3 %**	4096 carriles para 1521 sitios
Pallas sobre Triton, aplanado a 2048	**+21.0 %**	la mitad de carriles desperdiciados, pero solo **−2.9 %** de mejora: refuta que el desperdicio fuera la causa

Los tres **exactos** a redondeo de `f32` y los tres más lentos. El diagnóstico correcto fue: el costo es el **gather** — 1521 sitios × 7 vecinos × 3 componentes = **31 941 direcciones calculadas por carril**, contra siete lecturas contiguas coalescidas que produce `jnp.roll`.

6.2 El que funcionó, y por qué

Triton crudo, invocado desde JAX con `jax_triton.triton_call` — así el kernel vive en el **mismo programa XLA** que el anneal de producción y la comparación no es entre frameworks.

> **No hace menos lecturas: sigue emitiendo 21.** Cambia la **forma de la dirección**.
>
> Pallas calculaba **una dirección por carril**, así que el compilador no podía probar contiguidad y emitía accesos dispersos. Este kernel pide `xc + 1`: un **vector desplazado sobre un tramo contiguo**. Es el mismo patrón de acceso que produce un `roll`, pero el plano desplazado **nunca se materializa**: cae en registros, la actualización entera ocurre ahí y solo se escribe el resultado.

Dos detalles lo hacen posible:

- **El tablero de ajedrez hace innecesaria toda sincronización.** Todo vecino de un sitio actualizado tiene la **paridad opuesta**, así que ningún bloque lo escribe durante ese medio paso. El halo se lee de memoria global **sin barrera** — y esa es la verdadera razón de ser del tablero, que hasta entonces se tenía solo por corrección física.

- **El dato se reordena a `(B, 3, L, N, N)`**, con la componente como plano en vez de como eje más rápido, para que un tramo en `x` tenga paso 1. La transposición se paga una vez, fuera del bucle de pasos.

Después se llevaron también los **observables** a Triton: energía y magnetización como **parciales por bloque** (no atómicos, para que el resultado no dependa del orden en que terminan los bloques), y la carga topológica emitiendo los cuatro argumentos de sus dos arcotangentes para que JAX los cierre — porque Triton 3.8 **no tiene** `atan2` fuera de libdevice, y la rama importa: el denominador puede volverse negativo en una plaqueta frustrada.

6.3 La validación de física

Ningún tiempo se miró antes de pasar esto. Son cinco pruebas en dos niveles.

NIVEL 1 — EXACTITUD (NO HAY ESTADÍSTICA)

Se le dan a los dos motores **los mismos números aleatorios**, así la única diferencia posible es la aritmética. Cada espín debe salir idéntico:

prueba	resultado
stencil contra `sim_core.substep`, sublattice par	**0 de 182 520 espines distintos; `dE` a $1.4 \cdot 10^{-6}$**
idem, sublattice impar	**0 de 182 520**
observables de Triton contra los de JAX, mismo estado	**E a $1.2 \cdot 10^{-7}$ relativo, |M| a $2.0 \cdot 10^{-7}$, Q a $8.4 \cdot 10^{-8}$ absoluto**

NIVEL 2 — MUESTREO (ESTADÍSTICA)

Cada motor con **su propio** generador: Threefry contra Philox. Son cadenas de Markov distintas, así que no pueden coincidir espín a espín; lo que debe coincidir es la distribución.

prueba	resultado
E/sitio a equilibrio, T = 20, 5, 1	**0.09 – 0.23 σ**
norma del espín (la propuesta en la esfera)	**$1.000000 \pm 2.4 \cdot 10^{-7}$**
curvas reducidas completas (E, M, Cv, χ) en 24 temperaturas, contra el **control de producción contra sí misma**	**0.99 / 1.90 contra un control de 0.87 / 2.53 — o sea dentro del propio ruido de la vara**

El control es la pieza metodológica importante. Correr **producción contra sí misma** con otra semilla da el **ruido propio de la vara de medir**. Fue lo que destapó el bug del RNG y, tras arreglarlo, lo que lo cerró. Sin ese control, un desvío de 4.94 σ parecía culpa del kernel nuevo.

6.4 El bug del RNG, encontrado por el control

Qué pasaba. El kernel aceptaba 1.1 % menos propuestas que producción, y las cinco temperaturas caían del mismo lado. El nivel 1 pasaba perfecto —0 espines distintos— porque en esa prueba el RNG del kernel no se usa: se le pasan los números de JAX. Solo el nivel 2 podía verlo.

Cómo se acotó. Un diagnóstico a β→0, donde toda propuesta se acepta y por tanto la salida es la propuesta, descartó la propuesta: media por componente dentro de 1 σ de cero y `|s| = 1.000000`. Quedaba el uniforme.

La causa, mía. Sorteaba los tres gaussianos y el uniforme con claves Philox adyacentes (`sd+0` … `sd+3`) sobre un contador compartido. Eso le da al uniforme una entrada común con la dirección propuesta, así que correlaciona con `dE` y sesga el test de Metropolis a la baja. El patrón correcto de un generador por contador es el inverso: una clave, contadores distintos (`off·4 + j`).

Tras el arreglo. Las razones de aceptación se reparten a los dos lados de 1 (0.996, 0.998, 0.994, 1.029, 0.997), todas dentro de 1 σ, y la ventana corta pasó de 1.45/4.94 a 0.99/1.90, por debajo del propio control. Sin costo en velocidad.

6.5 El piso de lote, medido

El kernel pierde con lotes flacos. Con 12–13 réplicas por tarjeta va +47.7 % y +54.7 % más lento que producción, sobre cuatro millones de barridos y con los dos conteos concordando. Está construido para lotes anchos: con 12 réplicas su rejilla baja a unos 600 bloques de 128 hilos —unos 77 000 hilos para una tarjeta que quiere mucho más— mientras los kernels de XLA usan bloques mayores.

El cruce está entre 13 y 25 réplicas por dispositivo.

réplicas por dispositivo	H100	T4
12	**+47.7 %**	—
13	**+54.7 %**	—
25	−45.5 %	−58.7 %
50	−63.9 %	−57.0 %
125	−63.4 %	—
250	−66.9 %	−51.4 %
500	**−82.2 %**	−59.8 %
2500	−79.2 %	−58.5 %
5000	−78.1 %	**−49.6 %**

> **Y la ventaja no crece igual en toda tarjeta. Al ensanchar el lote, en H100 va de −63.9 % a −78.1 %; en T4 se encoge de −57.0 % a −49.6 %. La tasa del motor optimizado en T4 empeora monótonamente con el ancho (7.282 → 9.007 → 9.201 µs) mientras en H100 mejora y se estanca (0.873 → 0.677 → 0.677).**
>
> **Así que «más estados, mejor aprovechamiento» vale en H100 y no se traslada a la T4. La hipótesis razonable es la jerarquía de memoria —4 MB de L2 y 320 GB/s contra 50 MB y 3.35 TB/s, con 456 MB de conjunto de trabajo a 5000 réplicas— pero eso no se midió. Lo medido es el signo.**

6.6 Portabilidad: lo que bloqueó la T4 no era la T4

La primera corrida en T4 murió así: `ptxas fatal : Unsupported .version 8.8; current version is '8.7'`. No era la GPU ni el kernel — la T4 es capacidad de cómputo **7.5**, de sobra para Triton. Triton **trae su propio `ptxas`** (CUDA 12.9, ISA 8.8) y emite PTX para esa versión, mientras XLA ensambla con el de la rueda `nvidia-cuda-nvcc-cu12` (CUDA 12.8). Una versión menor de diferencia y no compila nada.

El arreglo: inventariar todos los `ptxas` de la máquina y fijarle a Triton el de XLA con `TRITON_PTXAS_PATH`, **antes de importar `jax`**, porque el backend fija su cadena al inicializarse. En Modal nunca apareció porque allí quedó `jax 0.10.2`; Kaggle resuelve `0.11.1`.

7 · Metodología, y los errores que costó aprenderla

Buena parte de esta sección no es física: es aprender a medir sin engañarse. Cada regla de abajo existe porque antes se violó.

7.1 Qué kernel se cronometra

> **Un bucle pelado de `substep` sub-reporta la ganancia. La misma búsqueda de particiones dio −1.3 % con un bucle de `substep` y −28.3 % con `eng["anneal"]`: un factor de veinte. XLA fusiona a nivel de programa, así que un paso aislado no compila igual que dentro del anneal completo. Regla: cronometrar siempre el kernel de producción.**

7.2 El sobrecoste de lanzamiento

Una llamada cuesta **~122 µs planos**, sin importar el tamaño de la rejilla. Nunca se cronometra un paso suelto: el bucle va dentro de `lax.fori_loop`. Eso es lo que destapó una falsa victoria —ver el registro— y es también por qué más olas empeoran el protocolo de partición.

7.3 El piso de resolución

La extrapolación desde celdas cortas trae **±2.5 %** de ruido por punto. Las estructuras por encima de ~5 % son reales; por debajo no se resuelven con esa celda. Este piso se violó dos veces y las dos costaron una conclusión equivocada.

7.4 La compilación fuera del cronómetro

Se compila con `.lower().compile()`, que compila **sin ejecutar**. Una versión anterior calentaba con una llamada real, lo que en una escalera de cuatro millones de barridos **duplica la factura** — y eso agotó una cuenta.

7.5 Puntuar las extrapolaciones en vez de confiar en ellas

Diez celdas extrapoladas se midieron después a longitud completa:

celda	extrapolado	medido	error
1× T4 estándar	55.9 min ×20	56.4	**+0.9 %**
1× T4 optimizado	25.7 ×20	24.3	−5.4 %
2× T4 estándar	28.4 ×20	28.4	**−0.1 %**
2× T4 optimizado	11.7 ×20	11.4	−2.7 %
1× H100 optimizado	22.57 ×5	22.58	**+0.04 %**
2× H100 estándar	35.82 ×5	35.80	**−0.06 %**
2× H100 optimizado	11.85 ×5	11.78	**−0.6 %**
4× H100 estándar	18.79 ×5	18.57	−1.2 %
4× H100 optimizado	6.87 ×5	6.33	**−7.9 %**
1× H100 estándar, 500 répl.	127.09 ×5	126.9 ×2.5	**−0.1 %**

Peor error 7.9 %, ocho de diez bajo el 3 %, y el ×20 acertó dentro del 5.4 %.

Nueve de diez erraban por exceso — las medidas salieron más rápidas. Y el peor error es la celda **más corta** (6.33 min), no la del factor más grande: **el error sigue a lo corta que es la celda en reloj, no al tamaño del factor**. Esa es la regla que salió de aquí.

7.6 Verificar la física en la corrida que produce el reloj

Un hueco que hubo que cerrar. El banco de tiempos cronometraba cuatro celdas y las reportaba sin evidencia propia de física: la verificación vivía en otra corrida, y el argumento se reducía a «el mismo archivo corrió allá y su suma SHA-256 coincide». Es una cadena de evidencia real, pero no es una medición. Ahora un paso `verify` corre primero y se niega a cronometrar nada si la física falla. Cuesta segundos contra decenas de minutos de reloj.

7.7 Un linter es más barato que una GPU

> **Dos corridas de Kaggle murieron por cosas que se ven en seco. Una porque el cuadernillo nunca escribía un módulo (un rango de celdas mal cortado), otra por pasar un argumento que la función no tenía (`So=So`). Hasta entonces solo se comprobaba que los archivos parsearan, y parsear solo prueba que es Python sintáctico. Ahora el generador corre pyflakes y compara los argumentos de cada llamada contra `inspect.signature` antes de empujar.**

7.8 Otras reglas que quedaron

- **Ventanas de lote contiguas.** Es lo único que hace visible la estructura fina; un barrido disperso la promedia y la esconde.
- **Identidad física del dispositivo registrada.** Las corridas viejas guardaron solo `"cuda:0"`, así que el control existía por diseño pero **no es auditable**. Las posteriores graban modelo y UUID.
- **Brazos intercalados** (63, 64, 65, 63, 64, 65, …), para que una GPU que se calienta reparta su deriva entre todos los brazos.
- **Pre-registrar una regla antes de medirla.** Se hizo con `B mod 8`: se escribió en git y luego puntuó 9 de 12. Sin pre-registro habría sido fácil recordarla como un acierto parcial.
- **Un par aislado es evidencia débil.** El par 64-vs-65 funciona cuando el efecto domina el dentado —la T4 con su +80 %— y falla cuando no: en H100 el mismo par mide +1.5 %, +1.8 % o +2.5 % con traslape, y según con qué se compare puede leerse +4.1 % o +14.5 %.

8 · Los números finales

Escalera de producción completa: 200 temperaturas × 20 000 barridos = 4 000 000 de barridos por réplica. «Estándar» es el motor de JAX de siempre; «optimizado» es el kernel de Triton.

8.1 Un estado (50 réplicas)

hardware	estándar	optimizado	cambio
CPU, 24 hilos	31.4 h	—	—
1× T4	56.4 min	24.3 min	−57.0 %
2× T4	28.4 min	11.4 min	−59.9 %
1× H100	8.1 min	**2.9 min**	−63.9 %
2× H100	4.8 min	2.6 min	−45.5 %
4× H100	3.6 min	**5.3 min**	**+47.7 %**

Las diez celdas de GPU son cronómetro a 200 temperaturas. La fila de 4 GPU es el piso de lote: 12 réplicas por tarjeta.

8.2 Diez estados (500 réplicas)

hardware	estándar	optimizado
CPU, 24 hilos	13.1 d	—
1× T4	12.4 h ×20	5.0 h ×20
2× T4	4.8 h ×20	2.3 h ×20
1× H100	2.1 h ×2.5	22.6 min
2× H100	35.8 min	11.8 min
4× H100	18.6 min	**6.3 min**

8.3 Cien estados (5000 réplicas)

hardware	estándar	optimizado
CPU, 24 hilos	**130.7 d**	—
1× T4	4.2 d ×100	2.1 d ×100
2× T4	2.6 d ×100	25.8 h ×100
1× H100	17.2 h ×100	3.8 h ×100
2× H100	8.9 h ×100	1.9 h ×100
4× H100	2.7 h ×100	**57.6 min ×100**

La escala por estados es la que funciona. El motor optimizado gana **1.10×** al pasar de 1 a 4 GPU con **un** estado, y **3.91×** con cien. Misma máquina y mismo kernel: lo único que cambia es que hay trabajo suficiente para llenar las tarjetas.

Y por eso las tarjetas extra se pagan con **más estados**, no repartiendo uno. Un estado por tarjeta es paralelo trivial: sin `pmap`, sin repartos y sin lotes flacos.

Dos lecturas que no favorecen al kernel y hay que decir. El −82.2 % a 500 réplicas por tarjeta son dos efectos sumados: el optimizado mejoró (0.873 → 0.677 µs) y el estándar empeoró (2.416 → 3.813) porque `v₂(500) = 2` le dispara su propio castigo. Parte de esa cifra es que 500 es un lote malo para el estándar.

Y el estándar aparenta escalar superlinealmente de 1 a 2 GPU (2.1 h → 35.8 min, un 3.55× con el doble de hardware), lo cual es imposible: `v₂(500) = 2` lo castiga y `v₂(250) = 1` lo libera. No escaló mejor: escapó de su propio castigo. Lo mismo de 2 a 4 GPU con 5000 réplicas.

8.4 El cronograma: 50 réplicas × 20k contra 100 × 15k

Pregunta aparte, medida en su propia página. El brazo de **50 × 20k** es **29.6 % más barato** —no 33 %, porque el costo por barrido sube 5.6 % al ser la mitad de barridos de medición en vez de un tercio— y entrega lo mismo.

Y la conclusión salió al revés de lo que parecía. El brazo barato **no degrada χ: la mide mejor**. El otro la subestima sistemáticamente donde τ es largo, y eso corre el pico 0.6–0.9 K.

Eso además es **un defecto del estándar actual**: la ventana de 5000 barridos submuestrea χ para el estado lines en toda la región de transición, no solo en la comparación.

8.5 · Cuántas réplicas hacen falta de verdad: τ

Todo lo anterior trata el número de réplicas como una variable libre de costo. No lo es: las réplicas existen para la estadística, y cuántas hacen falta lo decide el **tiempo de autocorrelación**.

τ es cuántos barridos hay que esperar para que una medición sea independiente de la anterior. Se calcula como `τ_int = ½ + Σρ(k)` con la ventana de Madras–Sokal (cortar en el primer `M ≥ 5τ`), por FFT.

τ no depende solo de la temperatura: depende del estado. Medido en los cuatro estados del conjunto de datos, va de **9** para el uniforme a **482** para lines. La ventana de 5000 barridos de medición **submuestrea χ** para lines en toda la región de transición.

Con eso, el desequilibrio real de reparto entre estados es **2.2×**, no el 50× que se afirmó primero con una sola temperatura. Un reparto proporcional a τ necesitaría **298 réplicas** en vez de 400, un 26 % menos.

8.6 · La convención de 20k barridos y 50 réplicas

Se buscó en la literatura si el cronograma del proyecto es defendible.

No existe un estándar. Los estudios publicados de redes de skyrmiones varían en órdenes de magnitud y cada uno declara el suyo: 10^5 + 10^5 con 3 corridas; 2×10^5 por espín con la mitad de termalización; y así. No hay una cifra canónica a la que apelar.

> **Pero el cronograma es defendible, y precisamente por las réplicas.** Con $\tau = 482$ para lines, una ventana de 6667 barridos de medición da apenas **10.4 muestras independientes por réplica**. Son muy pocas — y es exactamente por eso que hacen falta las 50 réplicas: ×50 da **519 muestras independientes**, que ya es una estadística razonable.
> El argumento correcto no es «20 000 barridos bastan», sino «20 000 barridos **por réplica** bastan **porque hay 50 réplicas**».

9 · Registro de errores y correcciones

Lista completa, en el orden en que ocurrieron. Se incluye porque un informe que solo trae los aciertos no permite juzgar los aciertos.

9.1 Errores de interpretación

#	lo que se afirmó	qué lo corrigió
1	«Hay un óptimo y la curva es convexa»	19 puntos contiguos: el vecino del mínimo cuesta +57 %
2	«Es el compilador, **no** el hardware»	falsa dicotomía: XLA decide, el sistema de memoria pone el precio
3	«El paso entre réplicas causa la inmunidad»	cierto pero vacío: `v₂(paso) = 2 + v₂(L)`, era `L` con otro nombre
4	«Elegí `L` divisible por cuatro y el lote deja de importar»	verificado en **una** GPU mientras se afirmaba en general; y la inmunidad cuesta pagar ancho 4 en todas las celdas
5	«XLA emite conjuntos de kernels distintos»	el volcado de PTX propio: firmas idénticas. El «+55 %» que lo sostenía era un artefacto de emparejar por nombre — las fusiones `_2` y `_3` intercambian identidad
6	«El desperdicio de carriles explica el costo del kernel»	predicho −50 %, medido **−2.9 %**
7	«El eje de estados aporta paralelismo»	leído en el código: un estado por GPU. Lo señaló el usuario
8	«Hay un desperdicio de 50× en el reparto por τ»	medidos los cuatro estados: es **2.2×**. La cifra vieja era de una sola temperatura
9	«BP + réplicas debería dejar de usarse»	sobre-afirmación: el ganador cambia con el número de GPU
10	«Los 8 puntos entre el paso y la escalera son el bucle de medición»	los kernels de observables recuperan **4.5** de los 8
11	«El 1.1 % de déficit de aceptación es 2 a 6 σ»	mal calculado: el error de una razón es `√2·√((1−p)/(N·p))`, hasta **3.4 %**. Eran ~1.6 σ. Lo que de verdad señalaba algo era la **prueba de signos** (cinco de cinco del mismo lado, p ≈ 3 %)
12	«La extrapolación ×20 de la T4 es un **piso** por estrangulamiento térmico»	medida a 200 T: acertó al 5 % y erró **por exceso**. La celda corta ya medía el reloj sostenido

#	lo que se afirmó	qué lo corrigió
13	«Los datos de H100 están todos medidos»	solo los de **un estado**; los de diez eran ×5

9.2 Predicciones que fallaron

#	predicción	medición
14	El protocolo de partición pierde 2.7 % contra `B = 201`	empató (+0.14 %)
15	`51+49` gana 5–7 % a `50+50`	empató; la anomalía de `B = 50` era ruido de celda corta, no una estructura
16	Regla `B mod 4` para ordenar lotes	**retirada: `tile_B` varía**
17	Regla `B mod 8 ∈ {5,7}`, **pre-registrada en git**	**9 de 12 — murió**
18	Regla `B mod 16`	**refutada con datos ya en disco, sin gastar una corrida**
19	Victoria de Triton del −6.5 %	**retirada: la medición de un paso era dominada por el despacho; amortizada sobre 200 pasos es +26.3 % más lenta**
20	Los observables en Triton valen +0.8 puntos (prueba de 4 temperaturas)	la escalera real dice **+4.5**. Con el número corto se habrían descartado

9.3 Errores de diseño experimental

#	error	consecuencia
21	Se controló **la cantidad equivocada**: fijar los sitios en 2125 fijó también la huella en 2.1 MB	el brazo medía en un régimen **donde el efecto no existe**; la red de referencia vive en 5.1 MB
22	La red de `L = 8` del experimento de la capa tonta tiene 4232 sitios	cae **entre** los dos regímenes, así que el ahorro que el truco iba a comprar puede ser menor que el que se midió
23	Identidad del dispositivo guardada solo como `"cuda:0"`	el control existía por diseño pero **no es auditable**: nadie puede verificar desde los datos qué GPU fue
24	`B = 50` figuraba como «~9.15 s» interpolado	medido de verdad: **9.79 s**
25	La ganancia del protocolo se aplicó a todos los brazos de GPU	el castigo depende del lote **por dispositivo**; lo señaló el usuario
26	Se cambió la fuente de la ganancia de `anneal` al kernel simplificado	la gráfica pasó a leer 14.7 min donde antes 12.5; lo detectó el usuario
27	El banco de tiempos no verificaba física en su propia corrida	el argumento se reducía a «el mismo archivo con la misma SHA-256 corrió en otra parte»; lo señaló el usuario

9.4 Errores de implementación que costaron una corrida

#	error	detalle
28	`KeyError: 'surf'`	la clave de la geometría es `"surface"`
29	`jnp.ascontiguousarray` no existe	en esa versión de JAX
30	`tl.math.atan2` no existe en Triton 3.8	solo en libdevice; se resolvió emitiendo los cuatro argumentos y cerrando en JAX
31	`triton_call` añade un puntero por cada `out_shape`	volver condicional una salida dejó `de_ptr` sin ligar: `'NoneType' object has no attribute '_unflatten_ir'`. La lista de argumentos debe quedar fija
32	`tl.randn4x` y una constante de semilla que desborda `int32`	el desempaquetado de la tupla falla; cuatro sorteos de un tensor son seguros
33	Rango de celdas mal cortado (`A[1:8]`)	el cuadernillo de Kaggle **nunca escribió** `triton_bench.py` y murió en su `import`
34	`So=So` donde la función no tenía ese parámetro	otra corrida de Kaggle muerta, **después** de instalar y pasar la prueba de humo
35	`PARTIAL.pop` borró el registro de un brazo medido	marcó como extrapolada una celda que sí se había cronometrado
36	Reemplazo de un bloque HTML cortado en su **primer** `</div>`	el bloque tenía `div` anidados, así que sobrevivió la cola con un párrafo ya refutado y un `div` huérfano
37	`git push | tail`	la tubería devuelve el estado de `tail` y enmascara el fallo del push
38	Parseo del estado de Kaggle leyendo la línea 1	que era un aviso de versión del CLI, no el estado
39	`id` del cuadernillo distinto del slug real	HTTP 409 al empujar
40	`sys.executable` para lanzar Modal	era el Python del sistema, sin el paquete instalado

9.5 Errores de presupuesto

Una cuenta agotada y una estimación 3× corta. Cada brazo de medición se ejecutaba dos veces —calentar y medir— y en una escalera de cuatro millones de barridos eso duplica la factura. La estimación fue de ~1 GPU-hora y el consumo real 3.2, agotando los $14.52 que quedaban. El arreglo fue compilar con `.lower().compile()`, que compila sin ejecutar: la corrida siguiente, de tamaño comparable, costó $5.73.

Tarifa derivada de lo facturado: $4.95 por GPU-hora de H100, incluyendo el arranque del contenedor. Esa cifra es la que después permitió decidir qué celdas medir y cuáles dejar extrapoladas.

9.6 El bug con consecuencias físicas

Uno solo de todos los anteriores afectó a la física, y está en la sección 6.4: las **claves Philox adyacentes sobre un contador compartido**, que sesgaban la aceptación un 1.1 % a la baja. Lo encontró el control de producción contra sí misma; el resto de pruebas lo dejaba pasar.

Y la lección que lo generaliza. Cuando una comparación estadística marca un valor atípico, **la primera cosa que hay que poner a prueba es la vara de medir**, corriendo la referencia contra sí misma. No empezar sospechando del código nuevo ni defendiéndolo.

10 · Lo que queda abierto

pregunta	estado
¿Cuál de los 12 candidatos de tile elige Triton?	**sin resolver. La minimización de carriles puntúa 36 %. Se elige compilando y leyendo la firma**
¿Qué dispara la rama `tb = B`?	**sin resolver. Hay dos bandas limpias, [59,63] y [121,127], 2.4–3.5× más planas; cinco predictores refutados**
El ~23 % residual del kernel pesado	**sin atribuir**
¿Por qué la ventaja del kernel **se encoge** en T4 al ensanchar el lote?	**el signo está medido; el mecanismo no. La hipótesis es la jerarquía de memoria, sin medir**
El umbral de huella (~5 MB) que enciende la regla	**predicción falsable pendiente: barriendo `n_active` a `v₂` fijo, el castigo debería encenderse cerca de 3233 sitios a `B = 60`**
El techo de XLA: deja **1.8×** sobre la mesa	cuantificado, y es parte de lo que el kernel propio recupera
El bucle de medición como siguiente objetivo de kernel	los observables en Triton recuperaron 4.5 de 8 puntos; el resto está en los ocho acumuladores y el `scan`
Nsight Compute en la GPU local	**imposible: la RTX 5070 Laptop es `sm_120` y el Nsight de apt (2024.1) precede a Blackwell**

11 · Inventario de la sección

carpeta	qué hay
`informe/`	este documento, en HTML y en PDF
`paginas/`	las **6** páginas interactivas de las que sale este informe: `resumen`, `lotes`, `operaciones`, `reparto`, `replicas`, `cronograma`
`codigo/`	`sim_core.py` (el motor de referencia), `triton_halo.py` (el kernel propio, 784 líneas), `triton_bench.py` (el banco portátil con el `verify`), `app.py` (los puntos de entrada de Modal), más los módulos de ablación y planificación
`cuadernillos/`	**14** notebooks, incluidos los de Kaggle autocontenidos que embuten los módulos con su SHA-256
`resultados/`	**74** JSON de curvas y mediciones en Modal, **16** de Kaggle, más la tasa de CPU

Sobre la duplicación. Las páginas están **copiadas**, no movidas: `docs/pages/` sigue siendo lo que sirve GitHub Pages, así que moverlas rompería el sitio vivo. Cuando se decida el destino final de la sección, se resuelve de un lado o del otro.

Cómo reproducir las mediciones

medición	comando
El paso, por lote	`modal run app.py::triton_halo`
La escalera, los dos motores	`modal run app.py::triton_full`
Un estado, 1/2/4 GPU	`modal run app.py::state_ladder --reps 50`
La regla del ancho contra el kernel	`modal run app.py::triton_width`
El control estadístico	`modal run app.py::triton_check`
Portabilidad y T4	los cuadernillos `kaggle_triton_*.ipynb`, empujados con `kaggle kernels push`

Compilación excluida del cronómetro en toda medición (`.lower().compile()`). Identidad física de cada GPU registrada en las corridas posteriores a 2026-10. Ventanas de lote contiguas. Brazos intercalados. Todas las cifras de este informe provienen de los JSON en `resultados/` y pueden rastrearse hasta la corrida que las produjo.